

HDOC Day-27

Pyramid / Diamond Patterns in C

Name: Yug Nirwal **SAP ID:** 590033354 **Subject:** C Programming **Date:** 20/09/2026

Aim: To write a menu-driven C program using a switch statement and nested loops to print different pyramid and diamond patterns. Each pattern is implemented in a separate parameterless function with no return value, without using arrays.

1. Full pyramid

Prints a centered full pyramid using asterisks.

Algorithm

1. Read the number of rows n .
2. Use an outer loop from 1 to n .
3. Print leading spaces for right positioning.
4. Print $(2*i - 1)$ asterisks in the current row.
5. Move to the next line.

Test-case table

Test case	Choice	Rows (n)	Expected result
TC1	1	4	Correct full pyramid with 4 rows
TC2	1	3	Correct full pyramid with 3 rows

2. Inverted pyramid

Prints a centered inverted pyramid using asterisks.

Algorithm

1. Read the number of rows n .
2. Run the outer loop from n down to 1.
3. Print increasing leading spaces.
4. Print $(2*i - 1)$ asterisks.
5. Move to the next line.

Test-case table

Test case	Choice	Rows (n)	Expected result
TC1	2	4	Correct inverted pyramid with 4 rows
TC2	2	3	Correct inverted pyramid

			with 3 rows
--	--	--	-------------

3. Diamond pattern

Prints a complete diamond. The middle row contains $2n - 1$ asterisks.

Algorithm

1. Read n for the upper half.
2. Print a full pyramid from row 1 to n .
3. Then run rows from $n-1$ down to 1.
4. Print spaces and $(2*i - 1)$ asterisks for the lower half.

Test-case table

Test case	Choice	Rows (n)	Expected result
TC1	3	4	Correct diamond pattern with 4 rows
TC2	3	3	Correct diamond pattern with 3 rows

4. Palindromic number pyramid

Prints numbers increasing from 1 to the row number and then decreasing back to 1.

Algorithm

1. Read the number of rows n .
2. For each row i , print leading spaces.
3. Print numbers from 1 up to i .
4. Then print numbers from $i-1$ back to 1.
5. Move to the next row.

Test-case table

Test case	Choice	Rows (n)	Expected result
TC1	4	4	Correct palindromic number pyramid with 4 rows
TC2	4	3	Correct palindromic number pyramid with 3 rows

5. Repeated-number pyramid

Prints the row number $2i - 1$ times in row i .

Algorithm

1. Read the number of rows n .

2. For each row i , print leading spaces.
3. Print the current row number i exactly $(2*i - 1)$ times.
4. Move to the next row.

Test-case table

Test case	Choice	Rows (n)	Expected result
TC1	5	4	Correct repeated-number pyramid with 4 rows
TC2	5	3	Correct repeated-number pyramid with 3 rows

C Program

```
#include <stdio.h>

void fullPyramid();
void invertedPyramid();
void diamondPattern();
void palindromicPyramid();
void repeatedNumberPyramid();

int main()
{
    int choice;

    printf("1. Full pyramid\n");
    printf("2. Inverted pyramid\n");
    printf("3. Diamond pattern\n");
    printf("4. Palindromic number pyramid\n");
    printf("5. Repeated-number pyramid\n");
    printf("6. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1: fullPyramid(); break;
        case 2: invertedPyramid(); break;
        case 3: diamondPattern(); break;
        case 4: palindromicPyramid(); break;
        case 5: repeatedNumberPyramid(); break;
        case 6: printf("Program ended.\n"); break;
        default: printf("Invalid choice.\n");
    }
    return 0;
}

void fullPyramid()
{
    int n, i, j;
    printf("Enter number of rows: "); scanf("%d", &n);
    for(i=1; i<=n; i++) {
        for(j=1; j<=n-i; j++) printf(" ");
        for(j=1; j<=2*i-1; j++) printf("* ");
        printf("\n");
    }
}

void invertedPyramid()
{
    int n, i, j;
    printf("Enter number of rows: "); scanf("%d", &n);
    for(i=n; i>=1; i--) {
        for(j=1; j<=n-i; j++) printf(" ");
        for(j=1; j<=2*i-1; j++) printf("* ");
        printf("\n");
    }
}

void diamondPattern()
{
    int n, i, j;
```

```

    printf("Enter number of rows: "); scanf("%d", &n);
    for(i=1; i<=n; i++) {
        for(j=1; j<=n-i; j++) printf(" ");
        for(j=1; j<=2*i-1; j++) printf("* ");
        printf("\n");
    }
    for(i=n-1; i>=1; i--) {
        for(j=1; j<=n-i; j++) printf(" ");
        for(j=1; j<=2*i-1; j++) printf("* ");
        printf("\n");
    }
}

void palindromicPyramid()
{
    int n, i, j;
    printf("Enter number of rows: "); scanf("%d", &n);
    for(i=1; i<=n; i++) {
        for(j=1; j<=n-i; j++) printf(" ");
        for(j=1; j<=i; j++) printf("%d ", j);
        for(j=i-1; j>=1; j--) printf("%d ", j);
        printf("\n");
    }
}

void repeatedNumberPyramid()
{
    int n, i, j;
    printf("Enter number of rows: "); scanf("%d", &n);
    for(i=1; i<=n; i++) {
        for(j=1; j<=n-i; j++) printf(" ");
        for(j=1; j<=2*i-1; j++) printf("%d ", i);
        printf("\n");
    }
}

```

Compilation and Execution

The program can be saved as hdoc27.c and compiled in Kali Linux using:

```
gcc hdoc27.c -o hdoc27
./hdoc27
```

The program was checked against the prepared test cases. The following terminal-style images show the test runs in the same simple Kali Linux look as the reference screenshot.

Test Results - Full pyramid

Test Case 1: choice = 1, n = 4

```

(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 1
Enter number of rows: 4
      *
     * * *
    * * * * *
   * * * * * * *
  * * * * * * * *

```

Test Case 2: choice = 1, n = 3

```

(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 1
Enter number of rows: 3
      *
     * * *
    * * * * *

```

Test Results - Inverted pyramid

Test Case 1: choice = 2, n = 4

```

(kali@kali)-[~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 2
Enter number of rows: 4
* * * * *
 * * * *
  * * *
   *

```

Test Case 2: choice = 2, n = 3

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 2
Enter number of rows: 3
* * * * *
  * * *
    *
```

Test Results - Diamond pattern

Test Case 1: choice = 3, n = 4

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 3
Enter number of rows: 4
      *
     * * *
    * * * * *
   * * * * * *
  * * * * *
 * * *
*
```

Test Case 2: choice = 3, n = 3

```

(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 3
Enter number of rows: 3
  *
 * * *
* * * * *
 * * *
  *

```

Test Results - Palindromic number pyramid

Test Case 1: choice = 4, n = 4

```

(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 4
Enter number of rows: 4
  1
 1 2 1
1 2 3 2 1
1 2 3 4 3 2 1

```

Test Case 2: choice = 4, n = 3

```

(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 4
Enter number of rows: 3
  1
 1 2 1
1 2 3 2 1

```

Test Results - Repeated-number pyramid

Test Case 1: choice = 5, n = 4

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 5
Enter number of rows: 4
  1
 2 2 2
3 3 3 3 3
4 4 4 4 4 4
```

Test Case 2: choice = 5, n = 3

```
(kali@kali) - [~/Desktop/cprog]
$ ./a.out
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
Enter your choice: 5
Enter number of rows: 3
  1
 2 2 2
3 3 3 3 3
```

Result

The menu-driven C program was written and tested successfully. All five pyramid/diamond patterns were produced correctly using nested loops, a switch statement, and separate parameterless functions without arrays.